

Sati-AI Engine: ภาคปฏิบัติการ

บันทึก

เอกสารฉบับนี้เป็นแนวคิดเชิงสถาปัตยกรรมลำดับที่ 2

ต่อยอดจาก "Sati-AI: หลักการ จำนวน 3 หน้า"

สถานะ: ยังเป็นต้นแบบบนกระดาษ ยังไม่ผ่านการทดลองจริง

จุดประสงค์

แปลงหลักการ "สติ-ปัสสัทธิ" ให้เป็นโครงสร้างโค้ด Python

ที่สามารถนำไปต่อยอดกับ LLM ได้จริงในอนาคต

แกนหลักในโค้ด sati_ai.py

****Path แยก 2 สาย****

- REFUSE: เมื่อผิดศีล

- FAST / DELIBERATE: เลือกตามระดับความเสี่ยง ไม่ใช่ AND

****Passaddhi - การระงับ****

ทุกประโยคต้องผ่านการวัด `confidence`

ต่ำกว่าเกณฑ์ = ตัดทิ้ง หรือ ตัดป้าย "ไม่แน่ใจ"

****Right Speech Gate****

กรองคำตอบสุดท้าย 4 ด้าน: ไม่โกหก ไม่ส่อเสียด ไม่หยาบ ไม่เพ้อเจ้อ

สถานะการพัฒนา

[✓] ออกแบบ Abstract Class

[] เชื่อมต่อกับ LLM จริง

[] ทดสอบและวัดผล

[] ปรับปรุงตามผลทดสอบ

โครงสร้างคำสั่ง:

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from enum import Enum
```

```
class Path(Enum):
```

```
    REFUSE = "sila_block"    # ศีล — ไม่ตอบ
```

```
FAST = "fast"
DELIBERATE = "deliberate"
```

```
@dataclass
class Claim:
    text: str
    evidence: list = field(default_factory=list)
    confidence: float = 0.0
```

```
@dataclass
class Response:
    claims: list[Claim]
    def render(self) -> str: ...
```

```
class SatiPassaddhiAI(ABC):
    def __init__(self, cfg):
        self.cfg = cfg          # threshold แยกออกมา calibrate ได้

    def process(self, user_input, history=None):
        signal = self.sati_eval(user_input, history) # สถิติต่อเนื่อง ไม่ใช่ครั้งเดียว

        # ขั้นศีล — มาก่อนเสมอ
        if signal.harm > self.cfg.harm_max:
            return self.compassionate_refusal(signal)

        # 2 แกนอิสระ ไม่ใช่ AND เดียว
        path = Path.DELIBERATE if signal.epistemic_risk > self.cfg.risk else Path.FAST
        resp = self.generate(user_input, path)

        # ปัสสัทธิ — ทุก path ไม่เว้น fast
        resp = self.passaddhi(resp)
        resp = self.samadhi_check(resp)          # ความสอดคล้องภายใน

        return self.right_speech_gate(resp, signal)

    def passaddhi(self, resp: Response) -> Response:
        for c in resp.claims:
            c.confidence = self.causal_trace(c)    # อธิบายปัจจัยตา
            if c.confidence < self.cfg.tau_drop:
                c.text = None                      # ตัดทิ้ง + log
            elif c.confidence < self.cfg.tau_hedge:
                c.text = self.mark_uncertain(c.text) # ไม่ลบ แต่บอกว่าไม่แน่ใจ
        return resp
```

```
def right_speech_gate(self, resp, signal):
    t = resp.render()
    t = self.no_falsehood(t)    # มุสสาวาท
    t = self.no_divisive(t)    # ปิสุณวาจา
    t = self.no_idle_talk(t)    # สัมผัสปลาดปะ — คมความยาว
    if signal.agitation > self.cfg.agitation:
        t = self.gentle_tone(t) # อนุสสาวาจา
        t = self.upekkha_guard(t) # กันไม่ให้กลายเป็นเคอชช
    return t
```